Revisiting the open-source Critical Line Algorithm of Bailey and López de Prado: speed, correctness and robustness

Thomas Schmelzer
Jebel Quant Research
thomas.schmelzer@gmail.com

November 8, 2026

Abstract

This note is a companion to the `cvxcla` software paper. Bailey and López de Prado (2013) published the first documented open-source implementation of Markowitz's Critical Line Algorithm (CLA) in a scientific language, as a readable reference rather than a fast solver, and through the Python 3 port in PyPortfolioOpt it remains widely used. We compare that port with `cvxcla`, a later implementation of the same algorithm, in three reproducible experiments. First, the runtime gap grows with problem size, from 2.8× at $n = 20$ to 418× at $n = 640$. It is explained mainly by the event search, which evaluates each candidate asset from its own linear system and so does about n times the necessary work per turning point. The explicit inverse that is usually blamed accounts for at most 1.6×. Any vectorised implementation removes most of the gap, so only a small part of it can be attributed to `cvxcla`'s design. Second, checked against an independent convex solver alone, the shipped code returns an incomplete frontier on 32 of 57 well-posed problems, missing portfolios by up to 0.13 in a weight, and does not terminate on one more. The cause is a single defect, stale blocked weights, which has also been reported independently to PyPortfolioOpt; a second defect only drops parameter labels. With a two-method fix, every frontier the code returns passes the same check, and at condition numbers of 10^{14} and above the corrected code is more accurate than `cvxcla`. Third, the shipped code raises errors or returns incomplete frontiers on degenerate inputs that `cvxcla` handles.

Keywords: critical line algorithm, efficient frontier, portfolio optimization, parametric quadratic programming, reproducibility, open-source software.
JEL classification: C61, C63, G11.

1 Introduction

The Critical Line Algorithm [4] computes the entire mean-variance efficient frontier exactly, as a finite list of turning points between which the optimal portfolio moves linearly. Bailey and López de Prado [1] (BLdP) made it widely available: their 2013 paper published a complete Python implementation at a time when the best-known reference implementation was the VBA-Excel code accompanying Markowitz and Todd [5]. Their code became the `CLA` class of PyPortfolioOpt [6], through which most practitioners use it today.

`cvxcla` [8, 9] is a later implementation of the same algorithm. It states each segment of the frontier as the KKT system of a parametric quadratic program and adds structured covariance backends and handling of degenerate cases. A first preprint of its software paper [8] already related the two in an appendix: it noted the per-candidate inverse, reported that PyPortfolioOpt misses some turning points, and gave a 39-line CLA written with the KKT conditions. The software paper no longer contains that comparison; this note takes it over and measures what it stated, and the 39-line CLA is Appendix A. It is new in four respects: it decomposes the

runtime gap factor by factor, separating the explicit inverse, the event formula and the Python interpreter; it checks both implementations against an independent solver on 90 problems, including degenerate and ill-conditioned ones; it attributes each wrong frontier to a defect; and it corrects that first account on one point, since the second defect deletes only copies of turning points that are kept (Section 5).

What is tested. We run PyPortfolioOpt 1.6.0's `CLA`, the version of the BLdP code that users install. It is the 2013 code ported from Python 2 to Python 3, with minor corrections; we write *BLdP* for both. The two defects of Section 5 appear verbatim in the 2013 listing: `getMatrices` reads the blocked weights from `self.w[-1]` after `solve` has set `w[i_in]=bi_in`, and `purgeExcess` compares with `mu<mu_` [1, Snippets 5 and 8, Appendix A.1].

Contributions.

- A decomposition of the runtime gap into the effects of the explicit inverse, the per-candidate event search, the Python interpreter and `cvxcla`'s own design (Section 4).
- A test of both implementations against an independent convex solver, which identifies one defect that loses turning points and one that only drops parameter labels, each with a small fix (Section 5).
- A test on degenerate and ill-conditioned inputs (Sections 6 and 7).

2 The two implementations

Both implementations trace the solutions $w(\lambda)$ of

$$\max_{w} \ \lambda\,\mu^{\top} w - \tfrac{1}{2} w^{\top}\Sigma w \quad \text{subject to} \quad \mathbf{1}^{\top} w = 1, \ \ \ell \le w \le u, \tag{1}$$

for λ from ∞ down to 0, and use the same convention for λ. In `cvxcla` the problem is stated term by term with a fluent builder (Listing 1). The same builder adds general equality rows $Aw = b$ (`.equality`), inequality rows $Gw \le h$ (`.inequality`) and a leverage cap (`.leverage`), and accepts a covariance operator in place of the matrix Σ; BLdP's problem class is the special case of box bounds and one budget row.

Listing 2 states the same problem in PyPortfolioOpt. Its `CLA` takes only box bounds; the budget is implied, and other constraints cannot be added. The turning points are traced as a side effect of `max_sharpe` (or `efficient_frontier`) and are available only as the internal attributes `w` and `ls`. On this instance both libraries return the same ten turning points, to 1.3×10^{-15}, and the same maximum Sharpe ratio.

The two implementations also share several design choices. Both start from the same greedy first turning point: every weight at its lower bound, then assets filled in descending order of expected return until the budget binds. Both end with an explicit turning point at $\lambda = 0$, departing from Niedermayer and Niedermayer [7], who continue to negative λ. Both locate the maximum-Sharpe portfolio by a one-dimensional search between the two turning points that bracket the discrete maximum, and both interpolate linearly between adjacent turning points.

They differ in how each step is computed (Table 1), and most of the differences follow from one deliberate choice. Bailey and López de Prado name the Karush–Kuhn–Tucker conditions as one way to treat the inequality-constrained problem [1, p. 173], and instead divide it into unconstrained problems on the free assets. Without multipliers on the bounds, the decision which blocked asset to free requires a separate solve for each candidate. The difference that matters most below is this event search. When the free set grows, BLdP evaluates each candidate asset separately: it forms that candidate's free block, inverts it with `np.linalg.inv`, and computes the λ at which the candidate would enter. This happens inside the loop over candidates. When

```python
import numpy as np
from cvxcla import CLA

rng = np.random.default_rng(0)
n = 10
mu = rng.uniform(0.0, 1.0, n)              # expected returns
X = rng.standard_normal((2 * n, n))
Sigma = X.T @ X / (2 * n)                  # covariance

cla = (
    CLA.problem(mu, Sigma)                 # objective of (1)
    .bounds(0.0, 1.0)                      # l <= w <= u
    .budget()                              # 1'w = 1
    .trace()
)

for tp in cla.turning_points:              # lambda from inf down to 0
    print(f"{tp.lamb:10.4g}", np.round(tp.weights, 3))

sharpe, w = cla.frontier.max_sharpe        # maximum-Sharpe portfolio
```

Figure 1: Problem (1) in `cvxcla` 2.7.1. The trace stores one turning point per change of the free set; on this instance there are ten, from $\lambda = \infty$ to $\lambda = 0$.

```python
from pypfopt.cla import CLA

# mu, Sigma as in Listing 1
cla = CLA(mu, Sigma, weight_bounds=(0.0, 1.0))  # budget implied
cla.max_sharpe()                           # traces the turning points

for lam, w in zip(cla.ls, cla.w):          # internal attributes
    lam = np.inf if lam is None else lam
    print(f"{lam:10.4g}", np.round(w.ravel(), 3))
```

Figure 2: Problem (1) in PyPortfolioOpt 1.6.0, on the data of Listing 1. The first stored λ is `None`, standing for $\lambda = \infty$.

the free set shrinks, the inverse is computed once, outside the loop. `cvxcla` solves the current segment once and reads every candidate's event off that solution.

3 Experimental setup

All experiments use PyPortfolioOpt 1.6.0 and numpy 2.4.6 (Accelerate) on Python 3.12, on an Apple M4 Pro. Each is a single script in the `cvxcla` repository that pins its dependencies:

- `experiments/bldp_gap.py`: runtime decomposition;
- `experiments/bldp_agree.py`: correctness;
- `experiments/bldp_fair.py`: runtime on BLdP's own problem class;
- `experiments/markowitz_todd.py`: BLdP's own example;
- `experiments/inverse_cla.py`: a minimal vectorised CLA, used to separate the effect of the algorithm's formulation from that of `cvxcla`'s design.

Table 1: Differences between the two implementations.

	BLdP	`cvxcla`
Covariance	Dense matrix, sliced per step	Operator interface with dense, Gram and factor backends
Linear algebra	Explicit inverse of the free block	Block-eliminated KKT solve through an $m \times m$ Schur complement
Event search	One linear system per candidate asset, $O(n)$ per turning point	One solve per turning point, all events read off it
Factor models	Not supported	Diagonal-plus-low-rank backend via the Woodbury identity, without forming Σ
Constraints	Box bounds and a budget	Box bounds, general $Aw = b$ and $Gw \leq h$, leverage caps
Degeneracy	A perturbation of one mean when all means are equal	Lowest-index tie-breaking, tolerances on events, explicit errors for infeasible problems

Runtime. The problems are long-only with a budget constraint and a $K = 10$ factor covariance, for n from 20 to 640. Each size uses 3 random seeds and 3 repeats, each in a fresh process. Ratios are medians over seeds; their spread is within $\pm 10\%$. The first runtime experiment uses `cvxcla` 2.6.0, the second 2.7.0.

Correctness. Both implementations receive the same μ, Σ, ℓ and u. Each trace is checked on its own against CVXPY 1.9.3 [3] with Clarabel 0.11.1 [2] at tolerance 10^{-12}; neither implementation serves as the reference for the other. The primary test reads neither λ nor the other implementation: a trace is *efficient* if every turning point and every chord midpoint of its polyline in weight space is a minimum-variance portfolio for its own expected return (CVXPY solves $\min \frac{1}{2} w^\top \Sigma w$ subject to $\mu^\top w \geq \mu^\top w_{\text{probe}}$ and the constraints; excess variance below 10^{-8} relative to $1 + v^\star$), and if the polyline runs from a maximum-return portfolio to the global minimum-variance one. A lost turning point makes a chord cut a corner of the frontier, so its midpoint has excess variance. A second test reads the path as a function of λ: a trace is *exact* if it is feasible to 10^{-9}, runs from $\lambda = \infty$ to 0, and has a relative objective gap below 10^{-8} at every turning point and segment midpoint. Finally, to locate disagreements, we compare the two traces in weight space: they give the *same set of portfolios* if every turning point of each lies within 10^{-8} in max-norm of the polyline through the other's turning points. There are 90 problems in eight families, over 3 seeds: generic long-only, shorting, a conditioning sweep from $\kappa = 10^2$ to 10^{16}, bounds that bind exactly, all-equal means, tied means, duplicated assets and rank-deficient sample covariances. The first four families contain 57 feasible, non-degenerate problems. The conditioning sweep probes numerical robustness; covariance matrices estimated from returns with $\kappa \geq 10^{12}$ would be regularised before use, not optimised over.

4 Runtime

4.1 Size of the gap

The ratio of BLdP's runtime to `cvxcla`'s grows with n without levelling off: 2.8× at $n = 20$, 16× at 80, 146× at 320 and 418× at 640. Over the last doubling, BLdP's runtime grows like $n^{3.7}$ and `cvxcla`'s like $n^{2.1}$. Parsing the shipped source confirms that the only `np.linalg.inv` inside a loop is in the candidate loop of the addition branch. At $n = 640$ a trace performs 321 inverses per turning point, about $n/2$.

4.2 Where the gap comes from

Table 2 splits the gap into factors, changing one thing at a time. Replacing each candidate's fresh $O(n_F^3)$ inverse by an $O(n_F^2)$ bordered update, while keeping the loop, gains at most 1.6×. Most of the gap comes from evaluating the $O(n)$ candidates one at a time: each evaluation re-slices Σ and recomputes the entry formula, which costs $O(n \cdot n_F)$ with or without an inverse. A profile of the shipped code agrees. The inverses take 8% of a trace at $n = 20$ and 41% at $n = 640$, the slicing 27–40%, and the entry formula and removal branch most of the rest.

Table 2: Factors of the runtime gap. Each column changes one thing; their product is the total.

n	inverse → bordered update, loop kept	per-candidate loop → vectorised search	minimal vectorised CLA → `cvxcla`	total
20	0.9×	12×	0.3×	2.8×
80	1.0×	37×	0.4×	16×
320	1.3×	86×	1.3×	146×
640	1.6×	141×	1.9×	418×

The second column of Table 2 changes two things at once: the formula used for each candidate, and the move from a Python loop to array operations. Table 3 separates them, with a variant of the minimal CLA that uses the vectorised formula but visits the candidates in a Python loop. It also adds the effect of fixing the defects of Section 5, which makes BLdP 4–9% slower because it keeps turning points the shipped code deleted. The gap against the corrected code is 3.5× at $n = 20$ and 150× at $n = 320$.

Table 3: Factors of the gap, separating the formula from the interpreter.

n	defect fixes	formula	interpreter	`cvxcla` vs. minimal	total
20	0.91×	11×	1.22×	0.23×	3.1×
80	0.92×	27×	1.50×	0.44×	16×
320	0.96×	83×	1.45×	1.24×	144×

The formula factor grows like $n^{0.96}$. BLdP derives each candidate's λ from that candidate's own bordered system, at a cost of $O(n_F^3 + n \cdot n_F)$, for $O(n)$ candidates per turning point. Reading every candidate's λ off one solve of the current segment, as $-\gamma_i/\delta_i$, costs $O(n)$ per candidate. This change in complexity accounts for almost all of the gap. The interpreter factor, the same arithmetic dispatched once per asset instead of once per step, stays between 1.2× and 1.5×. The Python loop itself is therefore not the main cost.

4.3 The share attributable to `cvxcla`

The formula factor is not specific to `cvxcla`: the minimal vectorised CLA achieves it as well. `cvxcla`'s own contribution is the last factor in Tables 2 and 3. It is about 4× slower than the minimal code at $n = 20$ and 1.2–1.9× faster from $n = 320$ on, with the crossover near $n = 250$. A profile shows where the time goes:

- *Checks* (feasibility and projection, held bounds, active rows, tolerances, tied starts) take 48% of a trace at $n = 20$ (1.3 ms) and 25% at $n = 320$ (26 ms).
- *Generality*, the event search over inequality rows that is idle on these problems, takes at most 1%.
- The *core* takes 79 ms at $n = 320$, less than the minimal code's whole trace (131 ms), because it uses a block-eliminated Cholesky solve rather than an explicit inverse.

Maintaining the inverse by rank-one updates instead of rebuilding it pays off at scale in both codes: it makes the minimal CLA 3.9× faster at $n = 640$, and `cvxcla` 2.5× faster with its `IncrementalDenseCovariance` backend, with no gain below $n = 80$. With matched linear algebra, `cvxcla` is 1.25× faster than the minimal CLA at $n = 640$ (204 ms against 247 ms) and scales as $n^{1.7}$ against $n^{2.3}$.

4.4 Environment

Both codes ran on the same machine and software stack, so the hardware cancels in every ratio. Single-threaded BLAS changes no timing by more than 7%. The shipped BLdP code at $n = 160$ takes about 1.43 s on Python 3.12, 3.13 and 3.14 alike, and 3.83 s under numpy 1.26.4. Newer software therefore narrows the gap rather than widening it.

5 Correctness on well-posed problems

BLdP's own example. BLdP validate their code on the 10-asset example of Markowitz and Todd [5], printing the data and the 10 turning points [1, Tables 1–2]. All three variants, shipped BLdP, BLdP with the fix below and `cvxcla`, reproduce that table to its three printed decimals, the maximum Sharpe ratio of 4.4535 at risk 0.2274 and the minimum-variance risk of 0.2052, and agree with each other to 3×10^{-14}. Along this frontier, however, the free set only grows, from 0 to 10 assets, so the step in which the first defect below acts is never taken: the example could not have revealed it.

Generic problems. Checked against the solver alone, the shipped BLdP code returns the efficient frontier on 24 of the 57 problems. On 32 a chord cuts a corner of the frontier, and on one ($\kappa = 10^{16}$) the code does not terminate; no error is raised. Table 4 breaks the result down by family. Even among plain long-only problems, 4 of 12 fail ($n = 10$ to 100). `cvxcla` passes the same check on 55 of the 57, failing two at $\kappa \geq 10^{14}$ (Section 7).

Comparing the two traces in weight space shows what goes wrong. On all 32 failing problems the shipped code skips turning points of `cvxcla`'s frontier, by between 2.7×10^{-4} and 0.13 in a weight. Every point it returns lies on that frontier: the code loses turning points but does not add wrong ones. The comparison finds lost turning points on five more problems, at $\kappa \geq 10^{10}$, where the corrected BLdP code agrees with `cvxcla`. There the skipped corners move the weights by about 0.02, but along near-null directions of Σ, so the excess variance stays below the tolerance of the independent test, which counts these five as efficient.

Table 4: Traces that are the efficient frontier, checked against the solver alone ("efficient"), and, for shipped BLdP, traces that give `cvxcla`'s set of portfolios.

	BLdP as shipped		BLdP with stale-weight fix	`cvxcla`
Family	efficient	as `cvxcla`	efficient	efficient
Long-only	8/12	8/12	12/12	12/12
Shorting	5/9	5/9	9/9	9/9
Conditioning	5/24	0/24	23/24	22/24
Binding bounds	6/12	6/12	12/12	12/12
Total	24/57	19/57	56/57	55/57

The exact check against the QP, which reads the path as a function of λ, gives a similar count (exact on 21 of 57) but mixes two different defects. Only the first one loses portfolios.

Stale blocked weights. When a free asset reaches a bound, `_solve` sets the working vector to `w[i] = bound`. The method `_get_matrices` then reads the blocked weights from `self.w[-1]`, the *previous* turning point, where asset i still has its old value. The new turning point is solved with the wrong blocked weights, and its budget is off by exactly $u_i - w_{\text{prev},i}$ (for example -4.1×10^{-2}). The method `_purge_num_err` then removes it as infeasible, and a true turning point is lost. All four long-only failures come from this defect. Reading the blocked weights from the working vector, a change to two methods, makes every frontier the code returns pass the independent check: 56 of the 57, the last being the cycle at $\kappa = 10^{16}$. That includes the two problems at $\kappa \geq 10^{14}$ where `cvxcla` itself fails (Section 7). On the other 54 it gives `cvxcla`'s set of portfolios. As paths in λ, 52 of the 57 are exact. The defect has been reported independently in PyPortfolioOpt issue #750, where it makes `max_sharpe` return a suboptimal portfolio in 22 of 300 random problems.

A purge without tolerance. The method `_purge_excess` removes a turning point whenever a later one has a strictly higher $\mu^\top w$. Where $\mu^\top w$ is flat, rounding can make it rise by about 10^{-14}. Along an exact trace, however, $\mu^\top w$ is flat only where the weights are constant, since $\mathrm{d}(\mu^\top w)/\mathrm{d}\lambda = \mu^\top(\mathrm{d}w/\mathrm{d}\lambda)$ vanishes only when $\mathrm{d}w/\mathrm{d}\lambda = 0$. Every point this purge removes is therefore a copy of a turning point that is kept. With the first defect fixed, it removes 34 points in 17 of the 57 problems, each within 1.1×10^{-11} of a kept point. No portfolio is lost, and `max_sharpe`, which interpolates in weights rather than in λ, is unaffected. What is lost is the λ at which the weights start to move again. We first described this defect as removing true turning points; we thank the reporter of #750 for showing otherwise. With a relative tolerance of 10^{-12} in the comparison as well, 56 of the 57 problems are exact as paths in λ, with gaps at most 7×10^{-13}. The remaining problem cycles at $\kappa = 10^{16}$.

With both changes, BLdP and `cvxcla` trace the same path to 10^{-8} on every problem except the two discussed in Section 7. The first change alone gives the same set of portfolios. The defects are in the implementation, not in the algorithm.

6 Degenerate inputs

`cvxcla` is exact on all 30 problems with all-equal means, tied means, duplicated assets or rank-deficient covariances. BLdP, with or without the two changes, behaves as follows.

- *All-equal means.* It raises `TypeError` on all 6 problems. The 2013 paper adds 10^{-5} to the last mean in this case; PyPortfolioOpt ships that line commented out. With it restored, the code still raises for $n = 20$, and for $n = 5$ traces the frontier of the perturbed problem, with λ scaled by 10^5 (gap $\approx 4 \times 10^{-5}$). The exact frontier is the single minimum-variance portfolio.
- *Tied means.* It raises `TypeError` on 7 of 9. When the denominator c of the event formula is exactly zero, `_compute_lambda` returns `None`, and the comparison with the previous λ fails under Python 3 (PyPortfolioOpt issue #738). In one case c was only close to zero and the trace continued along a wrong path: it starts at one vertex of the tied face and never frees the other tied assets (gap 0.25). `cvxcla` starts at the minimiser on that face.
- *Duplicated assets and rank-deficient* Σ. It raises `LinAlgError` on 8 of 9 duplicated problems (PyPortfolioOpt issue #737) and `TypeError` on the 3 problems with fewer observations than $n/2$. When an exact duplicate of the top asset enters later, the inverse of the singular free block (condition number 1.5×10^{16}) does not raise. Every later point misses the budget and is removed, so the frontier ends at $\lambda = 11.4$ instead of 0. With 30 observations for 40 assets, the corrected code is exact on 2 of 3 problems and deviates near $\lambda = 0$ on the third (gap 2×10^{-6}).

- *Infeasible bounds* ($\sum_i u_i < 1$). It raises `IndexError` while constructing the first turning point. `cvxcla` raises `InfeasibleProblemError`.

7 Conditioning and termination

With both changes applied, BLdP's explicit inverse is remarkably accurate: gaps are at most 4×10^{-13} up to $\kappa = 10^{14}$, with one cycle at 10^{16}. `cvxcla` is exact up to $\kappa = 10^{12}$. At 10^{14} and 10^{16} it has one problem each with a gap of 5×10^{-8} and 8×10^{-8}, on a segment near $\lambda = 10^{-10}$ where its weights differ from the QP solution by 2×10^{-3} along a near-null direction of Σ. Here the corrected BLdP agrees with the QP to 10^{-14}. As shipped, BLdP fails the check from $\kappa = 10^2$ on, because of the defects of Section 5.

BLdP compares λ values without a tolerance and its main loop has no iteration limit. With bordered updates, which change λ by about 10^{-15}, an asset can leave and re-enter at the same λ indefinitely: 5 of 18 runs of that variant did not terminate. The shipped code terminated on all 18 problems. This cycling is independent of the two defects: the bordered variant cycles on the same 9 of 30 problems with or without the changes.

8 Discussion

What the runtime comparison supports. BLdP set out to make the CLA available and verifiably correct on the classic 10-asset example, and never claimed speed. At $n = 20$ the gap is about 3×. The measurements support a narrow statement: *the reference implementation has a per-step complexity about a factor of n above what the algorithm needs.* This is a property of its formulation, not of the CLA or of Python, and mostly not of `cvxcla`'s design. A large speed-up over BLdP should not be attributed to `cvxcla` as such; its own contribution is the last factor in Table 3, together with its robustness and the features outside BLdP's problem class.

Assessment by criterion. Table 5 summarises the comparison. BLdP's lasting contribution is availability: it established the open-source baseline and is in wide use. Its explicit inverse is also accurate at extreme condition numbers once corrected. `cvxcla` is ahead on correctness, handling of degenerate inputs, problem scope and speed at scale, and validates against an independent solver rather than a single reference example. Its formulation, the KKT system of a parametric quadratic program, is the one used in the parametric-programming and homotopy literature, including the LASSO path [10]. It is also the shorter one: written with the KKT conditions, a complete CLA for BLdP's problem class takes 39 lines of NumPy (Appendix A), against about 180 for the trace in PyPortfolioOpt. The `cvxcla` package itself is larger, because of its checks, backends and constraint classes.

9 Limitations

The code run is the Python 3 port in PyPortfolioOpt 1.6.0. The two defects are in the 2013 listing verbatim, but some failures on degenerate inputs are specific to Python 3: there, comparing `None` with a number raises `TypeError`, while under Python 2, where `None` compares below every number, the 2013 code proceeds; what it then returns was not tested. All timings come from one machine and one problem family; other CPUs and BLAS builds, $n > 640$ (and $n > 320$ for Table 3) and memory use were not measured. Software older than numpy 1.26 and Python 3.12 could not be tested, because the packages no longer install there. Correctness was tested for $n \leq 100$ and only for a budget with box bounds, the class BLdP supports. Profile fractions were taken under cProfile, which inflates the cost of code with many Python calls, so the share of `cvxcla`'s checks is an upper estimate.

Table 5: Assessment by criterion.

Criterion	Stronger	Basis
Correctness	`cvxcla`	Stale blocked weights lose turning points in BLdP (Section 5).
Degenerate inputs	`cvxcla`	BLdP raises or truncates the frontier on tied, duplicated and rank-deficient problems (Section 6).
Termination	`cvxcla`	BLdP has no tolerance on λ and no iteration limit (Section 7).
Problem scope	`cvxcla`	General linear constraints, leverage caps, factor models.
Speed at scale	`cvxcla`	Mostly from the vectorised event search, available to any implementation; 1.2–1.9× from `cvxcla`'s design.
Small problems	comparable	`cvxcla`'s checks cost about 4× against a minimal implementation at $n = 20$; it is still faster than BLdP.
Extreme conditioning	BLdP, corrected	More accurate at $\kappa \geq 10^{14}$ (Section 7).
Availability	BLdP	First open-source CLA in Python; widely used through PyPortfolioOpt.
Code size	KKT formulation	39 lines of NumPy for BLdP's problem class, against about 180 for PyPortfolioOpt's trace (Appendix A); the `cvxcla` package is larger because of its checks and features.

10 Conclusion

The Critical Line Algorithm is fast when each step solves the current segment once, reads every event off that solution and compares events with a tolerance. BLdP's implementation differs from this mainly in its event search, which explains nearly all of the runtime gap. Its one defect that changes the result, stale blocked weights, has a two-method fix, now proposed to PyPortfolioOpt (pull request #776). With it, BLdP returns the correct turning points on every well-posed problem tested here except one at $\kappa = 10^{16}$. We hope these measurements, and the scripts that reproduce them, are useful to the maintainers and users of both packages.

Acknowledgements

We thank David H. Bailey and Marcos López de Prado for publishing their implementation, which made this comparison possible, the maintainers of PyPortfolioOpt, and the reporter of PyPortfolioOpt issue #750 for an independent analysis that corrected our account of the purge.

Disclosure

The author develops and maintains `cvxcla`. The experiments are designed to attribute the runtime gap and to report where BLdP is stronger, and all scripts are public so that the results can be checked independently.

Code availability

The code and experiment scripts are available at `https://github.com/cvxgrp/cvxcla` and archived on Zenodo under `10.5281/zenodo.22209208`.

References

[1] D. H. Bailey and M. López de Prado. An open-source implementation of the critical-line algorithm for portfolio optimization. *Algorithms*, 6(1):169–196, 2013. doi:10.3390/a6010169.

[2] P. J. Goulart and Y. Chen. Clarabel: an interior-point solver for conic programs with quadratic objectives. *Mathematical Programming Computation*, 2026. doi:10.1007/s12532-026-00320-7.

[3] S. Diamond and S. Boyd. CVXPY: a Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016. https://www.jmlr.org/papers/v17/15-408.html.

[4] H. M. Markowitz. The optimization of a quadratic function subject to linear constraints. *Naval Research Logistics Quarterly*, 3(1–2):111–133, 1956. doi:10.1002/nav.3800030110.

[5] H. M. Markowitz and G. P. Todd. *Mean-Variance Analysis in Portfolio Choice and Capital Markets*. Wiley, 2000.

[6] R. A. Martin. PyPortfolioOpt: portfolio optimization in Python. *Journal of Open Source Software*, 6(61):3066, 2021. doi:10.21105/joss.03066.

[7] A. Niedermayer and D. Niedermayer. Applying Markowitz's critical line algorithm. In J. B. Guerard, editor, *Handbook of Portfolio Construction*, pages 383–400. Springer, 2010. doi:10.1007/978-0-387-77439-8_12.

[8] T. Schmelzer. cvxcla: the Critical Line Algorithm with an operator interface in Python. SSRN preprint, 2026. doi:10.2139/ssrn.7583179.

[9] T. Schmelzer. cvxcla, version 2.7.1. Software, 2026. doi:10.5281/zenodo.22209208.

[10] T. Schmelzer and T. Hastie. The Critical Line Algorithm and the constrained LASSO: one curve, two literatures. arXiv:2609.25704, 2026. https://arxiv.org/abs/2609.25704.

A A complete CLA in 39 lines

Written with the KKT conditions, the CLA is short. Listing 3 is a complete CLA for BLdP's problem class, problem (1), in 39 lines of NumPy. Each segment costs one bordered solve for the free weights $w_F = a + \lambda b$ and the budget multiplier, and every event is read off that solve: a free weight reaches a bound at $\lambda = (u_i - a_i)/b_i$ or $(\ell_i - a_i)/b_i$, and a blocked weight is freed where its reduced cost $\gamma_i + \lambda\delta_i$ changes sign, at $\lambda = -\gamma_i/\delta_i$. This is the vectorised event search of Section 4. Events are compared with a relative tolerance, and a weight that leaves the free set is set to its bound before the next solve, so neither defect of Section 5 can arise.

On random 10-factor long-only problems with $n = 20, 40, \ldots, 640$ the listing returns `cvxcla`'s turning points $(21, 42, 81, 159, 323, 641)$ to 10^{-13}, in time of the same order as `cvxcla`'s dense backend. It refuses rather than mis-solves infeasible bounds, a tied largest expected return and a numerically singular free block; `cvxcla` handles the last two (Section 6). The test `tests/test_comparison_listing.py` runs the listing as printed here and checks these claims.

```
import numpy as np

def cla(mu, S, lo, up, tol=1e-9):
    """Turning points (lam, w) of max lam mu'w - w'Sw/2, 1'w=1, lo<=w<=up."""
    n, w, rest = len(mu), lo.astype(float), 1.0 - lo.sum()
    for i in np.argsort(-mu, kind="stable"):   # first turning point
        take = min(up[i] - lo[i], rest)
        w[i], rest = w[i] + take, rest - take
        if rest <= 0:
            break
    if rest > 0 or (n > 1 and np.partition(mu, -2)[-2] == mu.max()):
        raise ValueError("infeasible bounds, or a tied largest return")
    F, lam, last, path = np.arange(n) == i, np.inf, -1, [(np.inf, w.copy())]
    while True:                                # one KKT solve per segment
        B, m, one = ~F, F.sum(), np.ones(F.sum())
        K = np.block([[S[np.ix_(F, F)], one[:, None]], [one, 0.0]])
        rhs = np.c_[np.r_[-S[np.ix_(F, B)] @ w[B], 1 - w[B].sum()],
                    np.r_[mu[F], 0.0]]
        x = np.linalg.solve(K, rhs)            # w_F = a + lam b, nu likewise
        a, b = w.copy(), np.zeros(n)
        a[F], b[F] = x[:m, 0], x[:m, 1]
        gam, dlt = S @ a + x[m, 0], S @ b - mu + x[m, 1]  # reduced cost
        with np.errstate(divide="ignore", invalid="ignore"):
            hit = np.where(F, np.where(b < 0, up - a, lo - a) / b, -gam / dlt)
        live = np.where(F, abs(b) > tol,
                        np.where(w <= lo, dlt > tol, dlt < -tol) & (lo < up))
        if last >= 0:
            live[last] = False                 # the asset that just flipped
        hit = np.where(live & (hit <= lam * (1 + tol)), hit, -np.inf)
        k = int(np.argmax(hit))                # ties: lowest index
        if hit[k] <= 0:
            return [*path, (0.0, a)]
        lam, w = hit[k], a + hit[k] * b
        if np.any((w < lo - 1e-8) | (w > up + 1e-8)):
            raise np.linalg.LinAlgError("free block numerically singular")
        if F[k]:
            w[k] = up[k] if b[k] < 0 else lo[k]  # leaving weight on its bound
        F[k], last = not F[k], k
        path.append((lam, w.copy()))
```

Figure 3: A complete CLA for box bounds and one budget row, written with the KKT conditions.